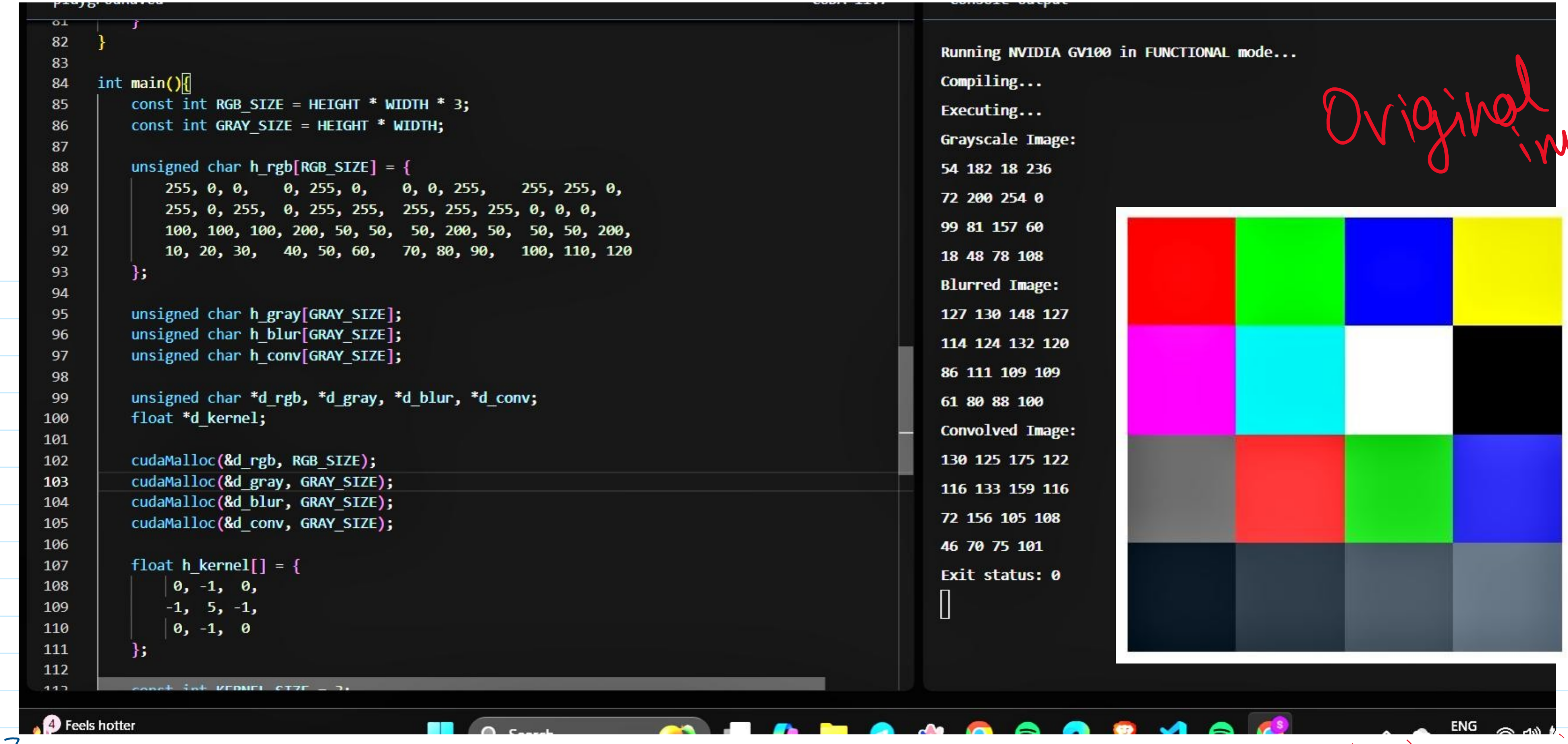
RGB_GRAYSCALE,BLUR, CONVOLUTION

27 May 2025 15:54



```
}

int main(){
    const int RGB_SIZE = HEIGHT * WIDTH * 3;
    const int GRAY_SIZE = HEIGHT * WIDTH;

    unsigned char h_rgb[RGB_SIZE] = {
        255, 0, 0,    0, 255, 0,    0, 0, 255,    255, 255, 0,
        255, 0, 255,  0, 255, 255,  255, 255, 255, 0, 0, 0,
        100, 100, 100, 200, 50, 50,  50, 200, 50,  50, 50, 200,
        10, 20, 30,   40, 50, 60,   70, 80, 90,   100, 110, 120
    };

    unsigned char h_gray[GRAY_SIZE];
    unsigned char h_blur[GRAY_SIZE];
    unsigned char h_conv[GRAY_SIZE];

    unsigned char *d_rgb, *d_gray, *d_blur, *d_conv;
    float *d_kernel;

    cudaMalloc(&d_rgb, RGB_SIZE);
    cudaMalloc(&d_gray, GRAY_SIZE);
    cudaMalloc(&d_blur, GRAY_SIZE);
    cudaMalloc(&d_conv, GRAY_SIZE);

    float h_kernel[] = {
         0, -1,  0,
        -1,  5, -1,
         0, -1,  0
    };
```

```
Running NVIDIA GV100 in FUNCTIONAL mode...
Compiling...
Executing...
Grayscale Image:
54 182 18 236
72 200 254 0
99 81 157 60
18 48 78 108
Blurred Image:
127 130 148 127
114 124 132 120
86 111 109 109
61 80 88 100
Convolved Image:
130 125 175 122
116 133 159 116
72 156 105 108
46 70 75 101
Exit status: 0
```

Original image

RGB to grayscale

gray = $0.299 \cdot R + 0.587\, G + 0.114\, B$

Pixel layout : of a 3x2 image (3 columns, 2 rows)

$[\ [R_{00}, G_{00}, B_{00}],\ [R_{01}, G_{01}, B_{01}],\ [R_{02}, G_{02}, B_{02}],$

$[R_{10}, G_{10}, B_{10}],\ [R_{11}, G_{11}, B_{11}],\ [R_{12}, G_{12}, B_{12}]\ ]$

It is stored like this

as it has value range from 0 to 255 which is similar to pixel value range (0–255)

unsigned char img[ ] = {

R_{00}, G_{00}, B_{00}, // pixel at (0,0)
R_{01}, G_{01}, B_{01}, // pixel at (0,1)
R_{02}, G_{02}, B_{02} // pixel at (0,2)
R_{10}, G_{10}, B_{10} // pixel at (1,0)
R_{11}, G_{11}, B_{11} // pixel at (1,1)
R_{12}, G_{12}, B_{12} // pixel at (1,2)

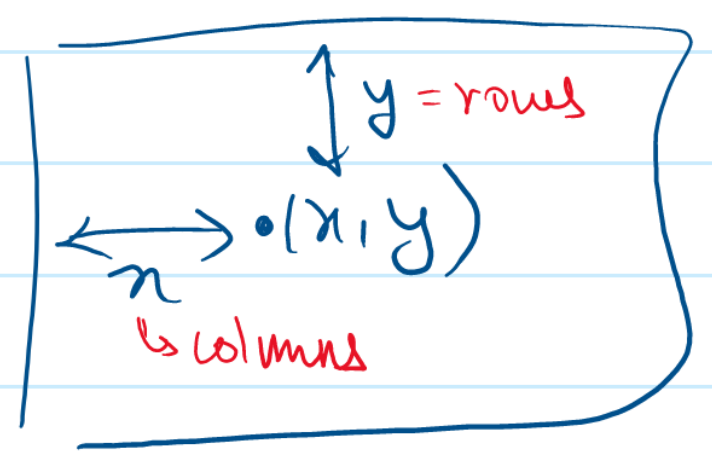
}

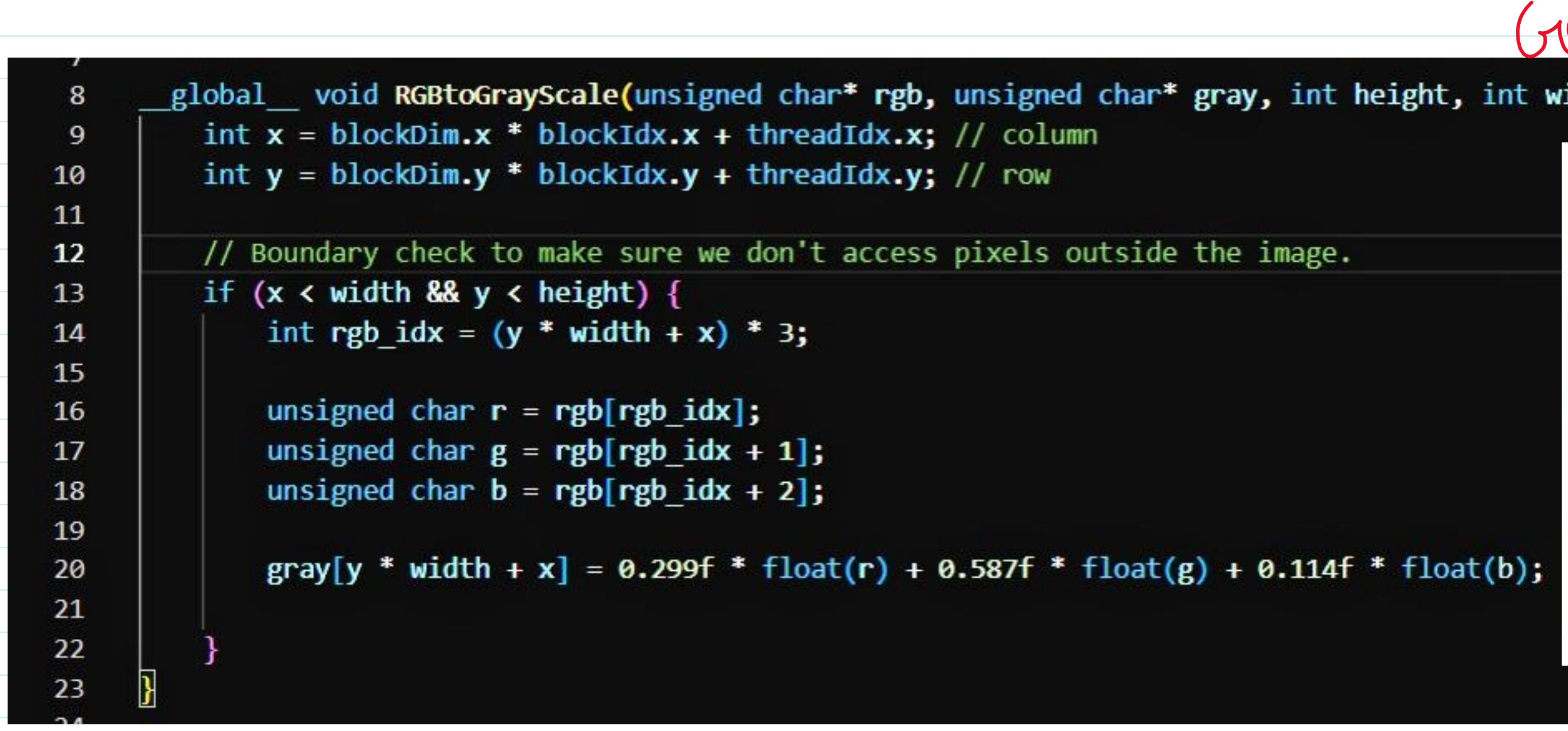
index = (i * width + j) * 3
unsigned char r = img[index]
unsigned char g = img [index+1]
unsigned char b = img [index+2]

eg- for pixel at (0,2)
i=0 , j=2
index= 6
r = img[6]
g = img [7]
b = img [8]

Boundary check to make sure we don't access pixels outside the image

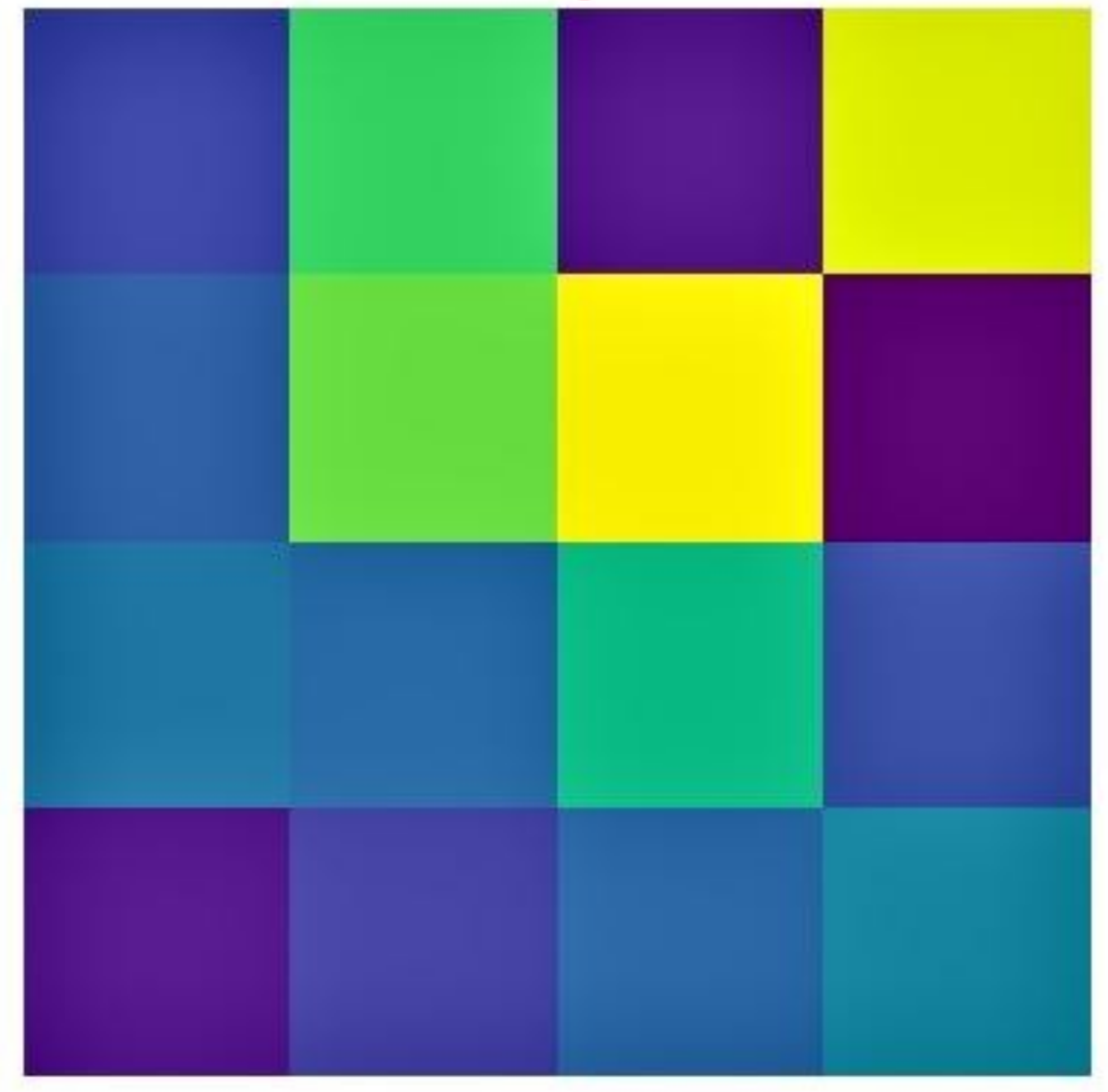
x < width && y < height



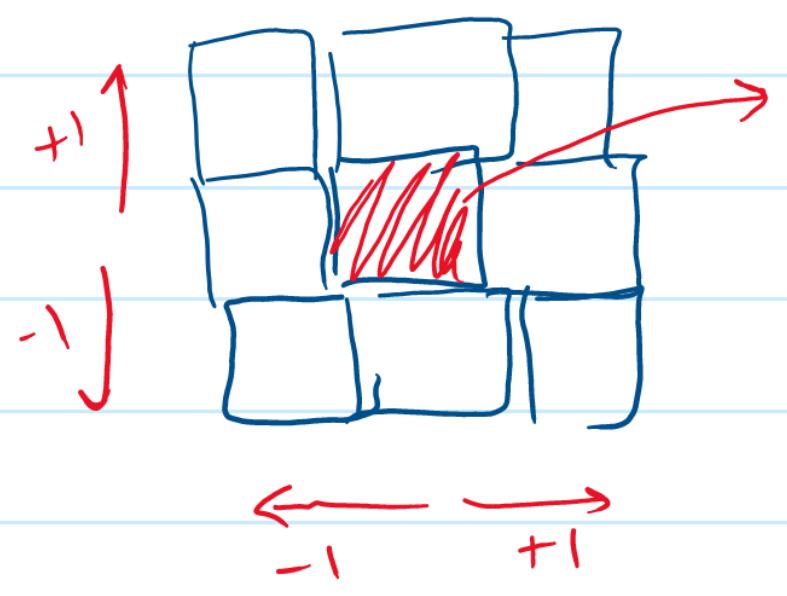


```
__global__ void RGBtoGrayScale(unsigned char* rgb, unsigned char* gray, int height, int width){
    int x = blockDim.x * blockIdx.x + threadIdx.x; // column
    int y = blockDim.y * blockIdx.y + threadIdx.y; // row

    // Boundary check to make sure we don't access pixels outside the image.
    if (x < width && y < height) {
        int rgb_idx = (y * width + x) * 3;

        unsigned char r = rgb[rgb_idx];
        unsigned char g = rgb[rgb_idx + 1];
        unsigned char b = rgb[rgb_idx + 2];

        gray[y * width + x] = 0.299f * float(r) + 0.587f * float(g) + 0.114f * float(b);

    }
}
```

Grayscale img



Blur



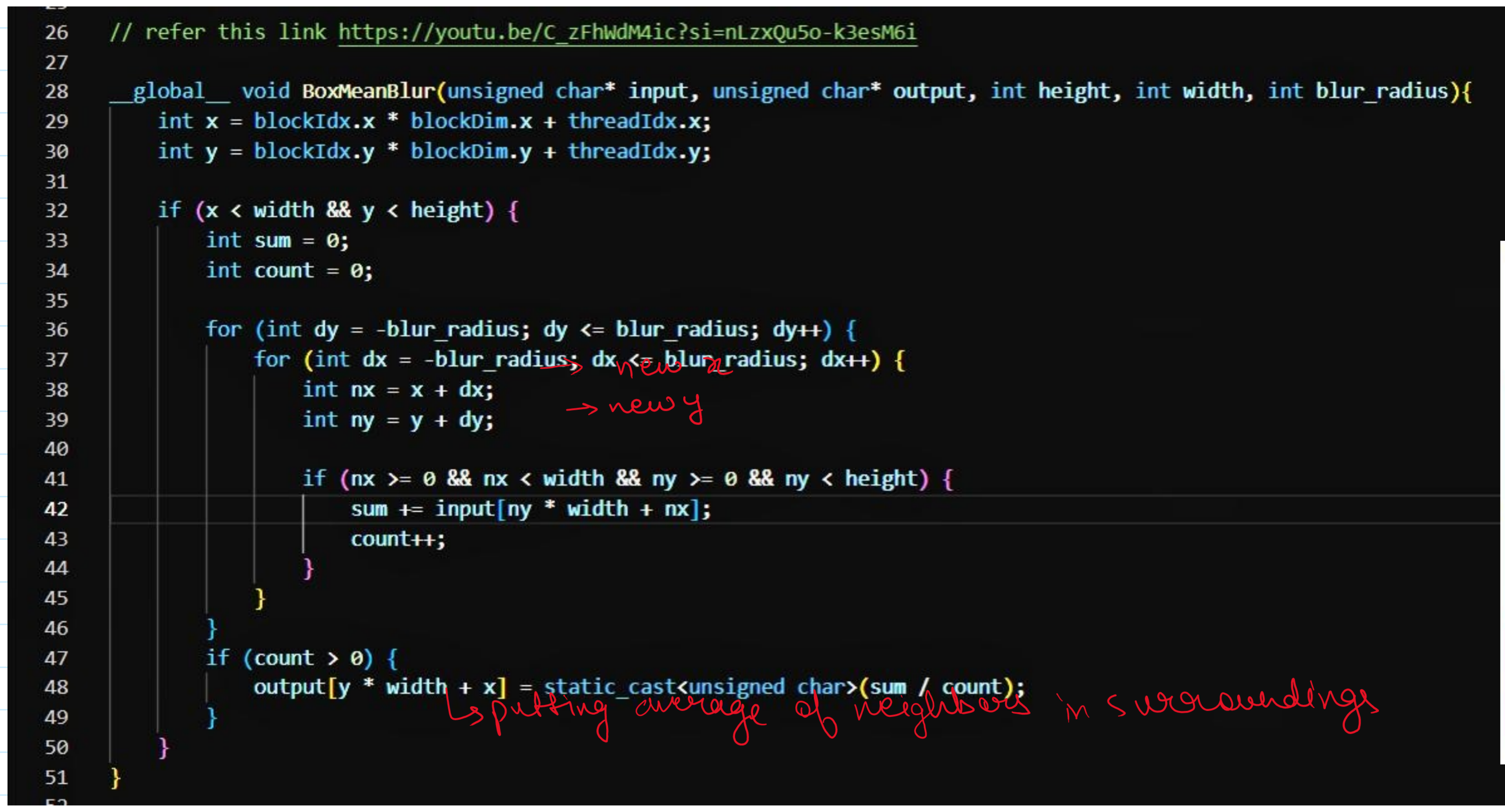
let blur radius be = 1
then for this centre pixel we will take sum of all the surrounding pixels & then get the average, & replace the cell value with the average

if centre pixel is (x,y)

then we go from x-1 to x+1 & y-1 to y+1 covering all pixels & taking it's average

x-1, y-1 → img [y-1 x width + x-1]
x-1, y
x-1, y+1
x, y-1
x, y
x, y+1
x+1, y-1
x+1, y
x+1, y+1

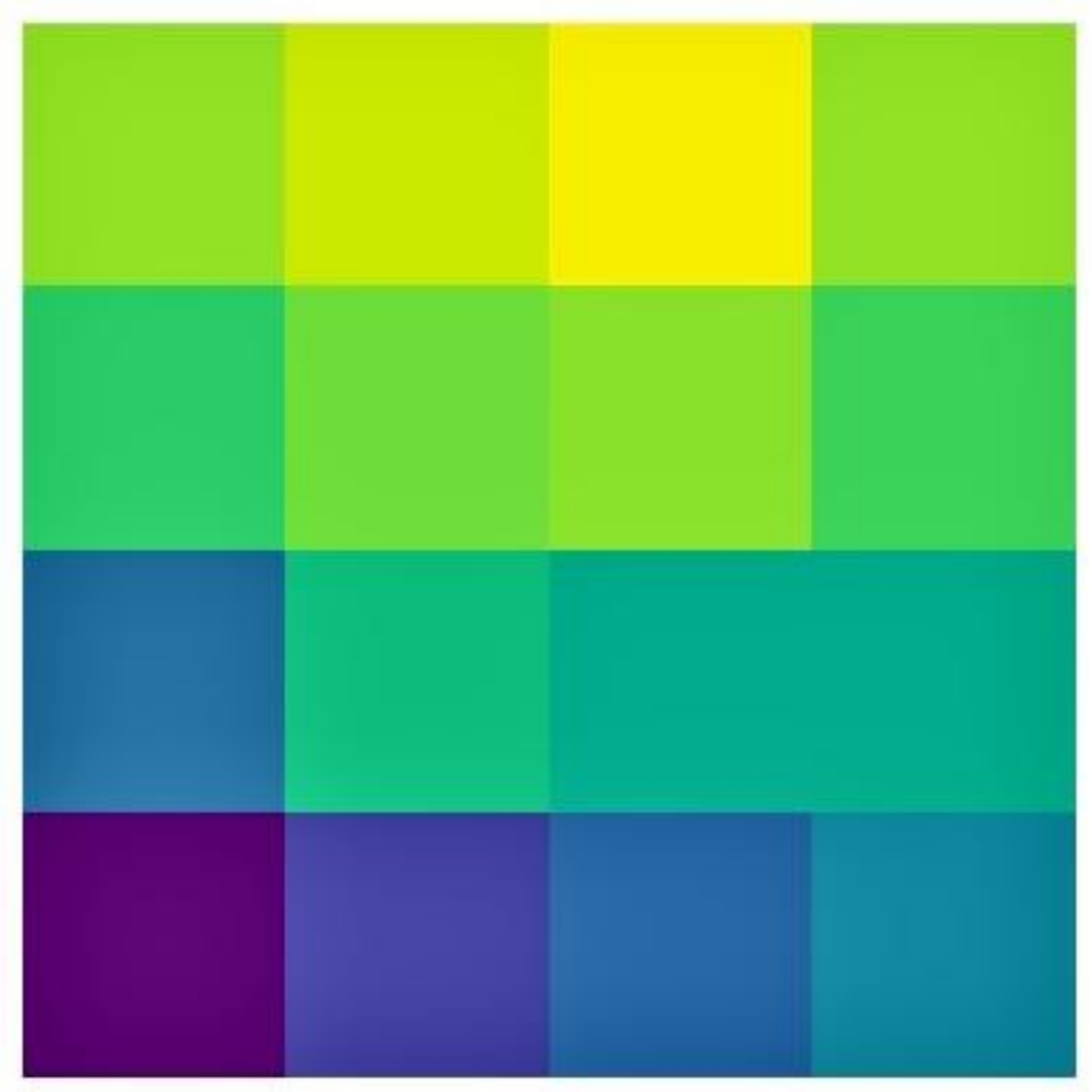
again we should make sure that our original pixel (x,y) is in image size & also the new pixels we are accessing are also in image



```
// refer this link https://youtu.be/C_zFhWdM4ic?si=nLzxQu5o-k3esM6i

__global__ void BoxMeanBlur(unsigned char* input, unsigned char* output, int height, int width, int blur_radius){
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;

    if (x < width && y < height) {
        int sum = 0;
        int count = 0;

        for (int dy = -blur_radius; dy <= blur_radius; dy++) {
            for (int dx = -blur_radius; dx <= blur_radius; dx++) {
                int nx = x + dx;
                int ny = y + dy;

                if (nx >= 0 && nx < width && ny >= 0 && ny < height) {
                    sum += input[ny * width + nx];
                    count++;
                }
            }
        }
        if (count > 0) {
            output[y * width + x] = static_cast<unsigned char>(sum / count);
        }
    }
}
```

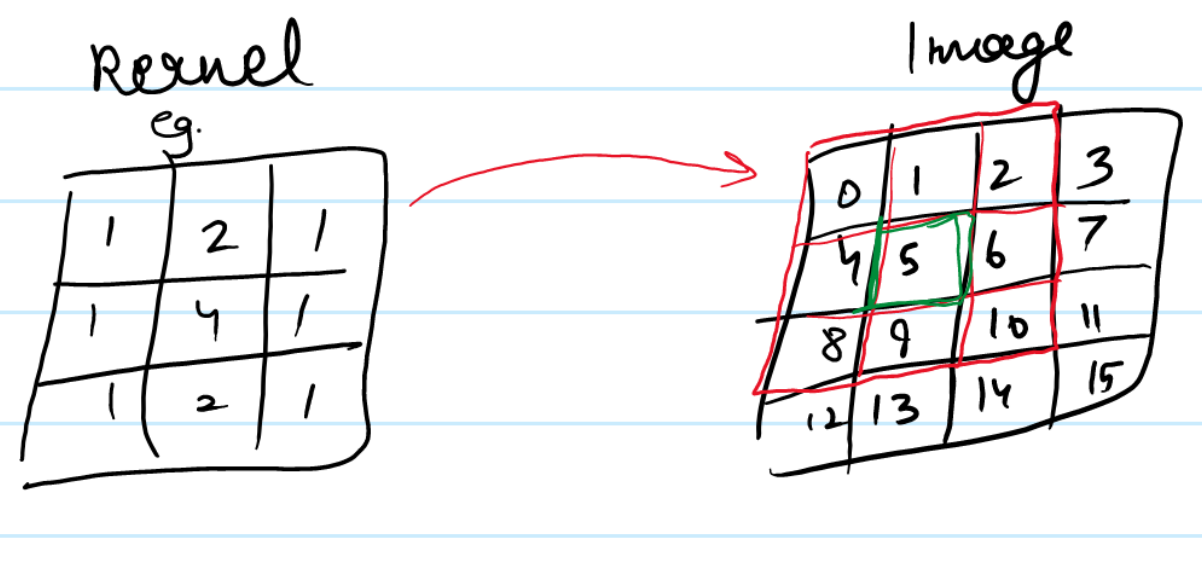
→ new x
→ new y
↳ putting average of neighbours in surroundings

Blur img



Convolving

We need a kernel which we can convolve over our image



$$\begin{bmatrix} 1 & 2 & 1 \\ 1 & 4 & 1 \\ 1 & 2 & 1 \end{bmatrix} \qquad \begin{bmatrix} 0 & 1 & 2 & 3 \\ 4 & 5 & 6 & 7 \\ 8 & 9 & 10 & 11 \\ 12 & 13 & 14 & 15 \end{bmatrix}$$

like (1x0 + 2x1 + 1x2)
(1x4 + 4x5 + 1x6) ⇒ 4+30+36 ⇒ 70 → Sum
1x8 + 2x9 + 1x10

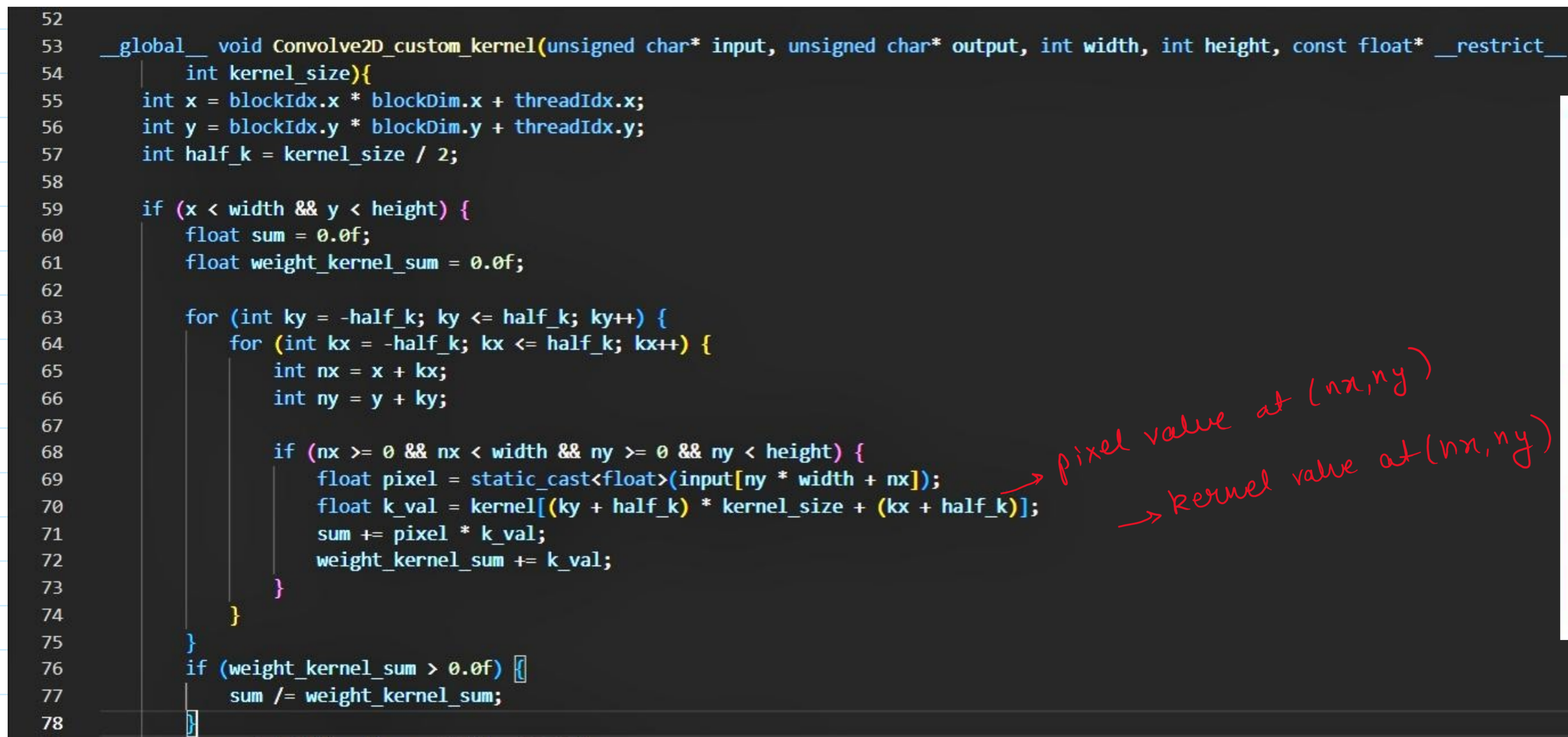
We now divide it with the kernel sum, Since our pixel was in a good position so entire kernel was able to convolve over it, otherwise if a corner or border pixel would have been chosen then, the entire kernel sum would not be taken, only those which overlap would have been taken.

Kernel sum = 1+2+1+1+4+1+1+2+1 = 14

In the end

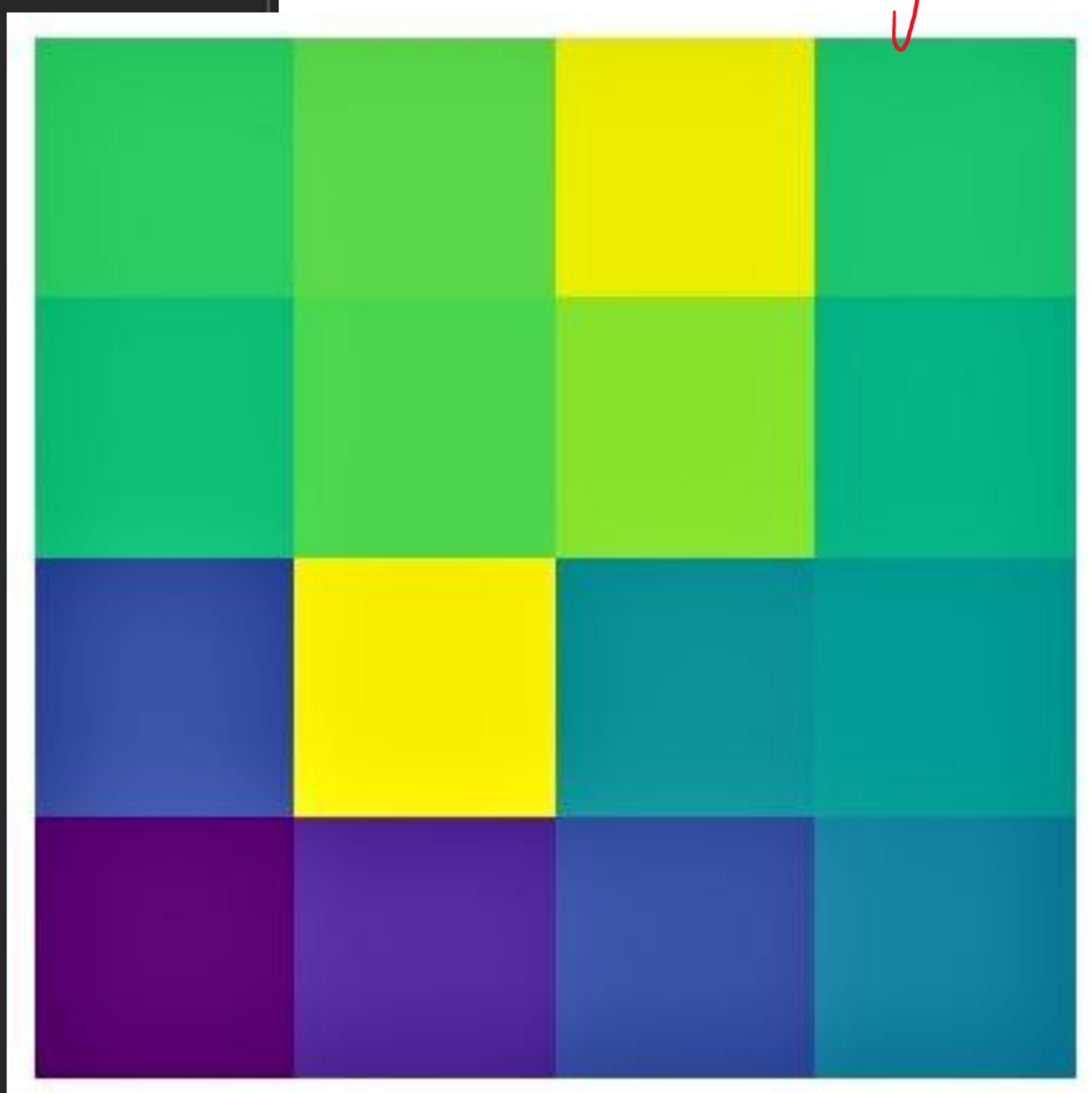
output [y* width +x] = Sum / kernel sum $= \frac{70}{14}$ ⇒ 5 ⇒ new cell value

In one way the blur kernel, is just convoluting the image with a kernel filled with all ones



```
__global__ void Convolve2D_custom_kernel(unsigned char* input, unsigned char* output, int width, int height, const float* __restrict__ kernel,
        int kernel_size){
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;
    int half_k = kernel_size / 2;

    if (x < width && y < height) {
        float sum = 0.0f;
        float weight_kernel_sum = 0.0f;

        for (int ky = -half_k; ky <= half_k; ky++) {
            for (int kx = -half_k; kx <= half_k; kx++) {
                int nx = x + kx;
                int ny = y + ky;

                if (nx >= 0 && nx < width && ny >= 0 && ny < height) {
                    float pixel = static_cast<float>(input[ny * width + nx]);
                    float k_val = kernel[(ky + half_k) * kernel_size + (kx + half_k)];
                    sum += pixel * k_val;
                    weight_kernel_sum += k_val;
                }
            }
        }
        if (weight_kernel_sum > 0.0f) {
            sum /= weight_kernel_sum;
        }
```

→ pixel value at (nx,ny)
→ kernel value at (nx,ny)

Convoluted img